



Bike System

Carrera	TECNICATURA SUPERIOR en DESARROLLO DE SOFTWARE		
Ciclo Lectivo	2026	Año	3°
Unidad Curricular	Prácticas Profesionalizantes II	Docente	Kappes Nicolas
Nombre y apellido	German Domingues- Exequiel Rios	Fecha	30-06-2026
Calificación			

Instituto Tecnológico El Molino (ITEC)

Desarrollo de Software

Prácticas Profesionalizantes II

Bike System

Domingues German - Rios Exequiel

Kappes Nicolas

Ciclo lectivo 2026

Fecha de entrega: 30-06-2026

índice	1
Maqueta de todas las pantallas del sistema.	2
Modelo lógico de base de datos.	8
Sentencias DDL de Bike System.	9

Maqueta de todas las pantallas del sistema.

Pantalla principal de inicio de sesión del programa.

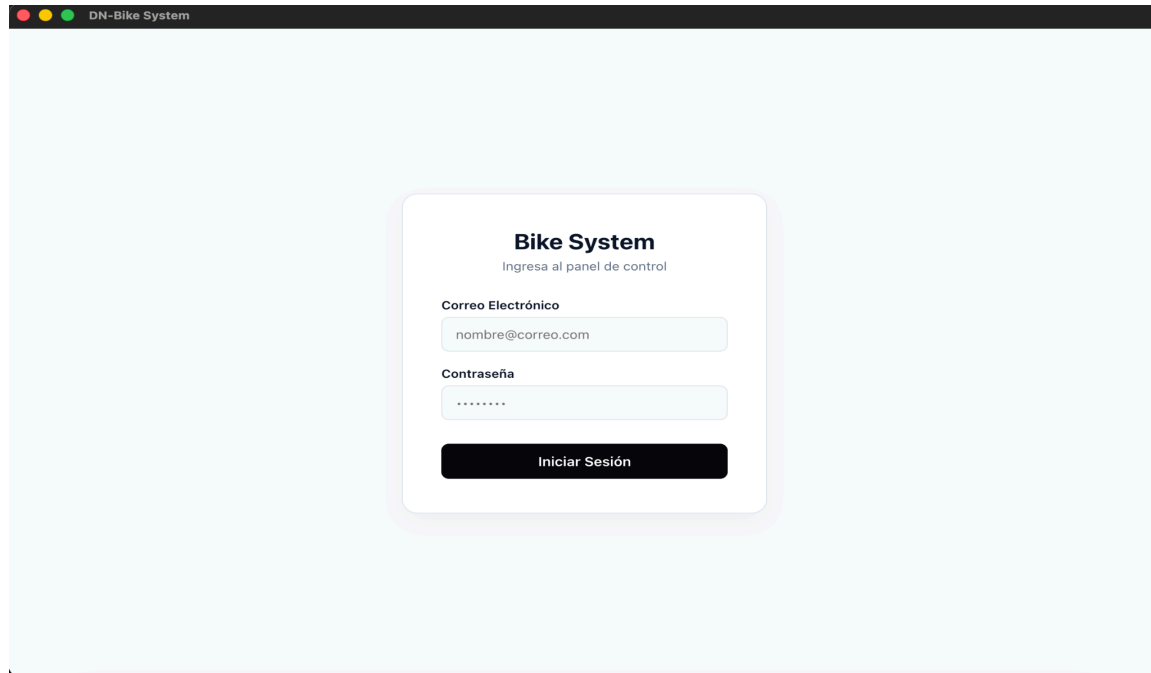


Figura 2: correspondiente al inicio de sesión(login).

Pantalla de dashboard principal.

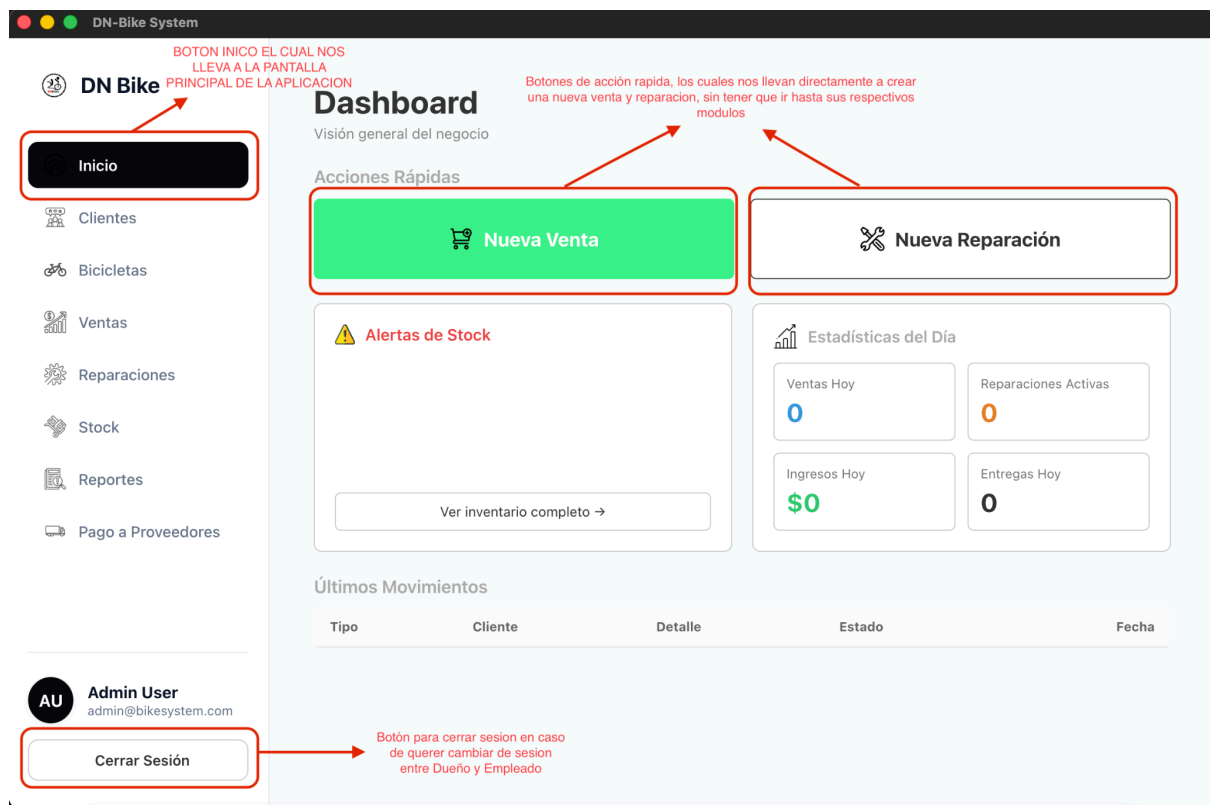


Figura 3: Pantalla del módulo de Inicio(Dashboard).

Pantalla módulo cliente.

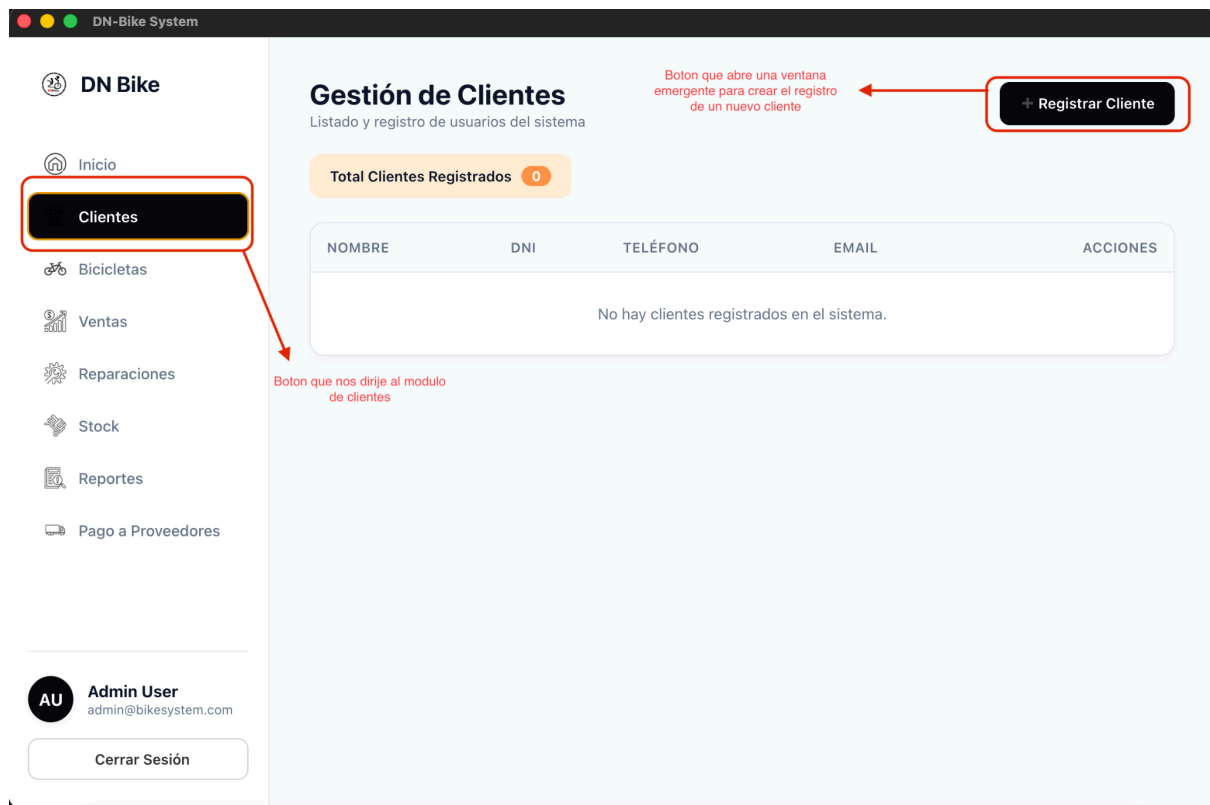


Figura 4: pantalla del módulo gestión de clientes.

Ventana emergente de nuevo cliente.

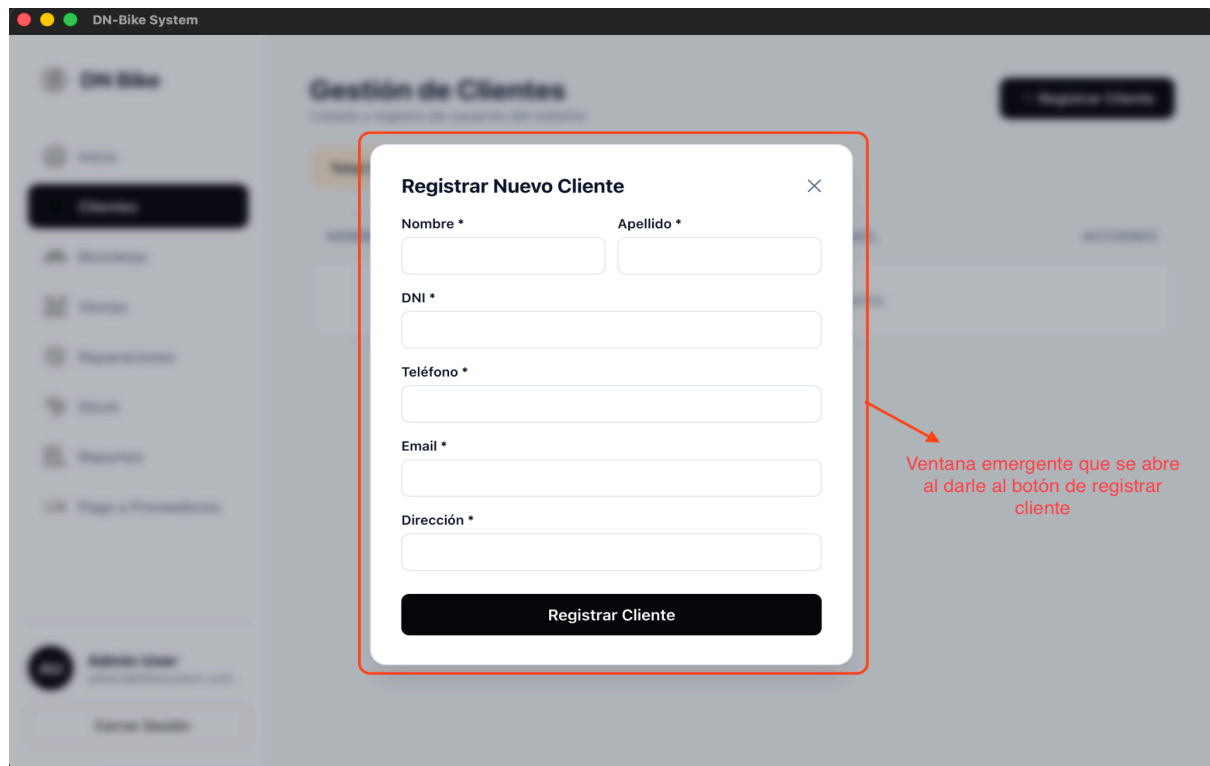


Figura 5: Ventana emergente para el registro de un nuevo cliente.

Módulo de Bicicletas.

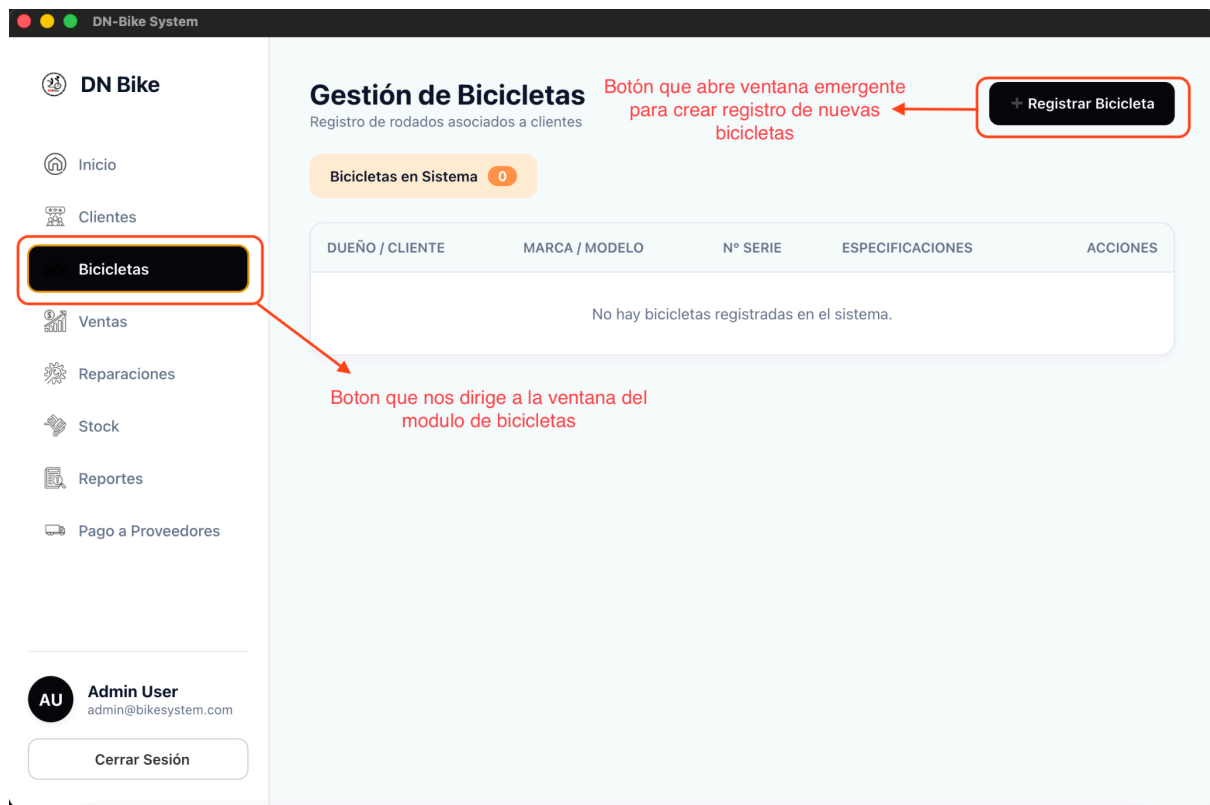


Figura 6: Pantalla del módulo de Gestión de bicicletas,

Ventana emergente para registrar bicicleta.

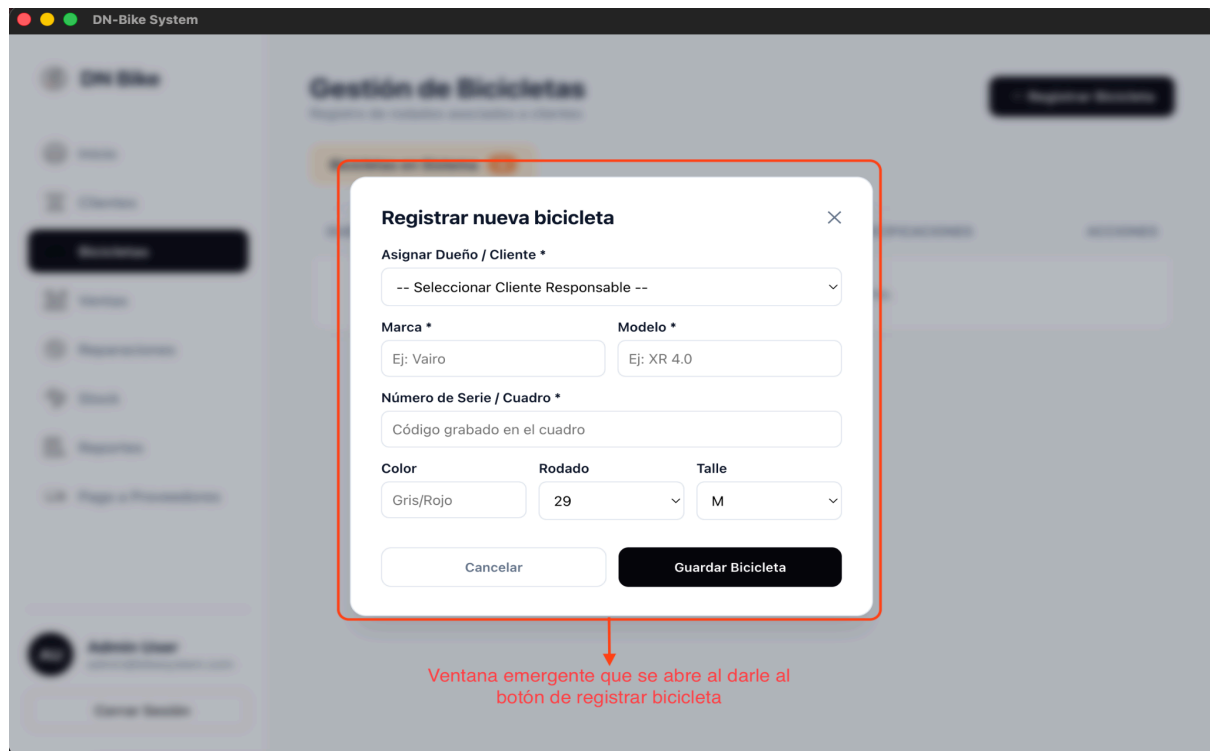


Figura 7: Ventana emergente para el registro de una nueva bicicleta.

Modulo de ventas.

Figura 8: pantalla



del módulo de ventas.

Módulo de Reparaciones.



Figura 9: Pantalla del módulo de Gestión de reparaciones.

Ventana emergente de Reparaciones.

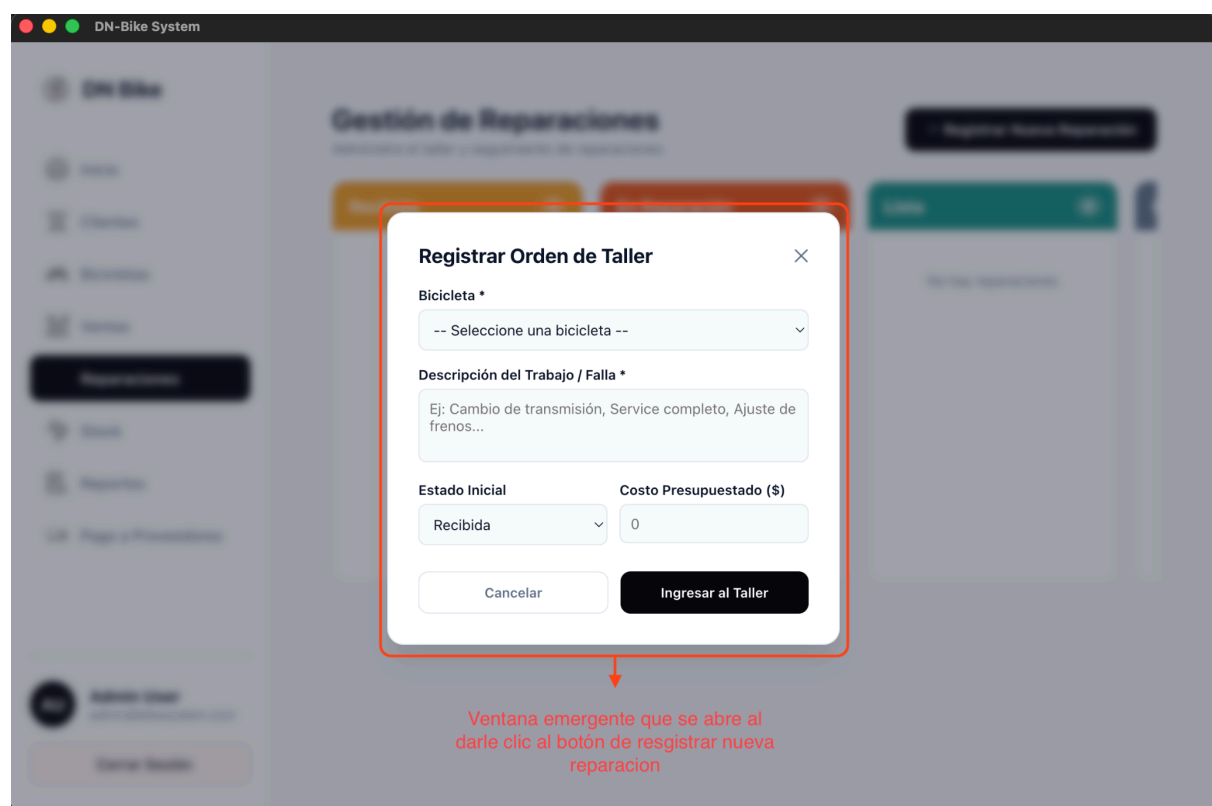


Figura 10: Ventana emergente para registrar una orden de taller (Reparaciones).

Módulo de stock.

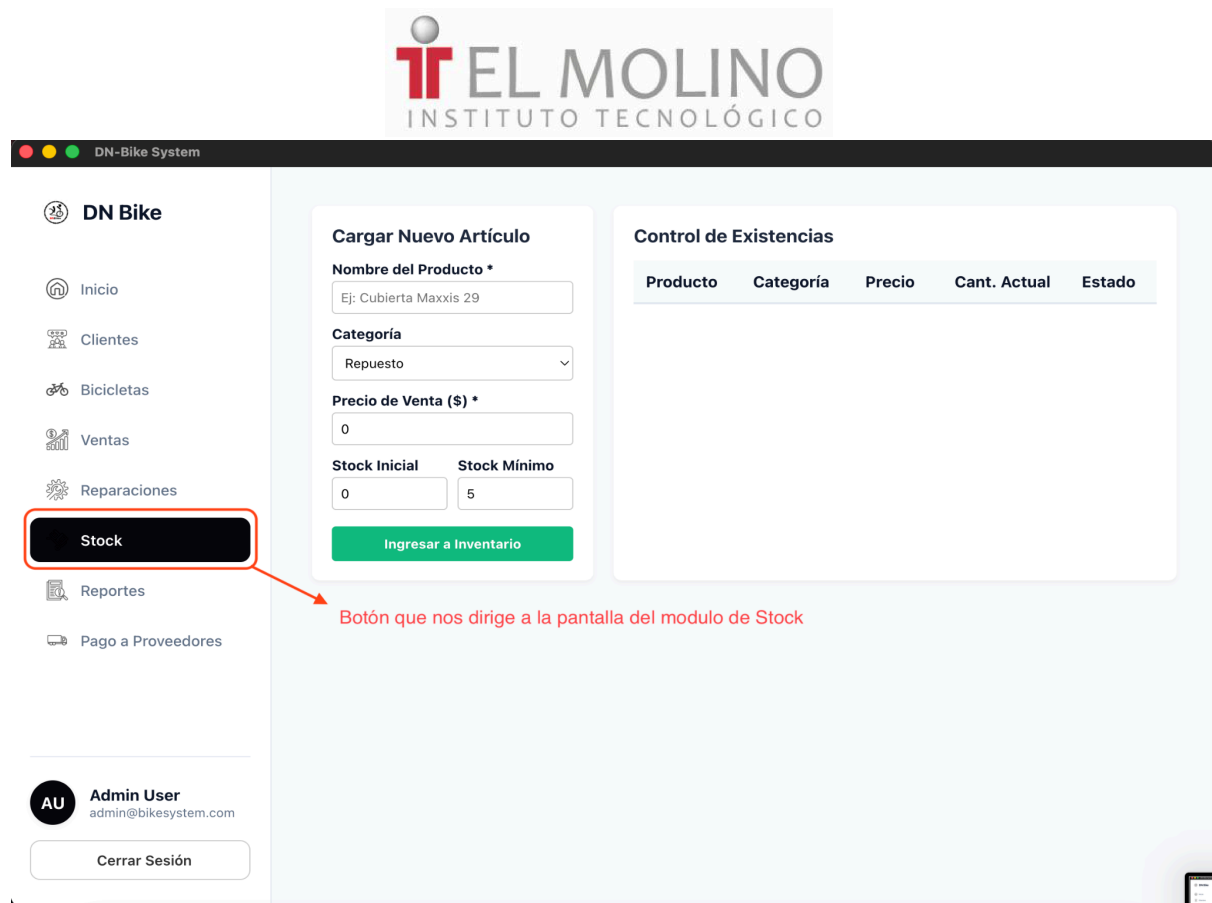


Figura 11: Pantalla del módulo de Control de Stock.

Módulo reportes.

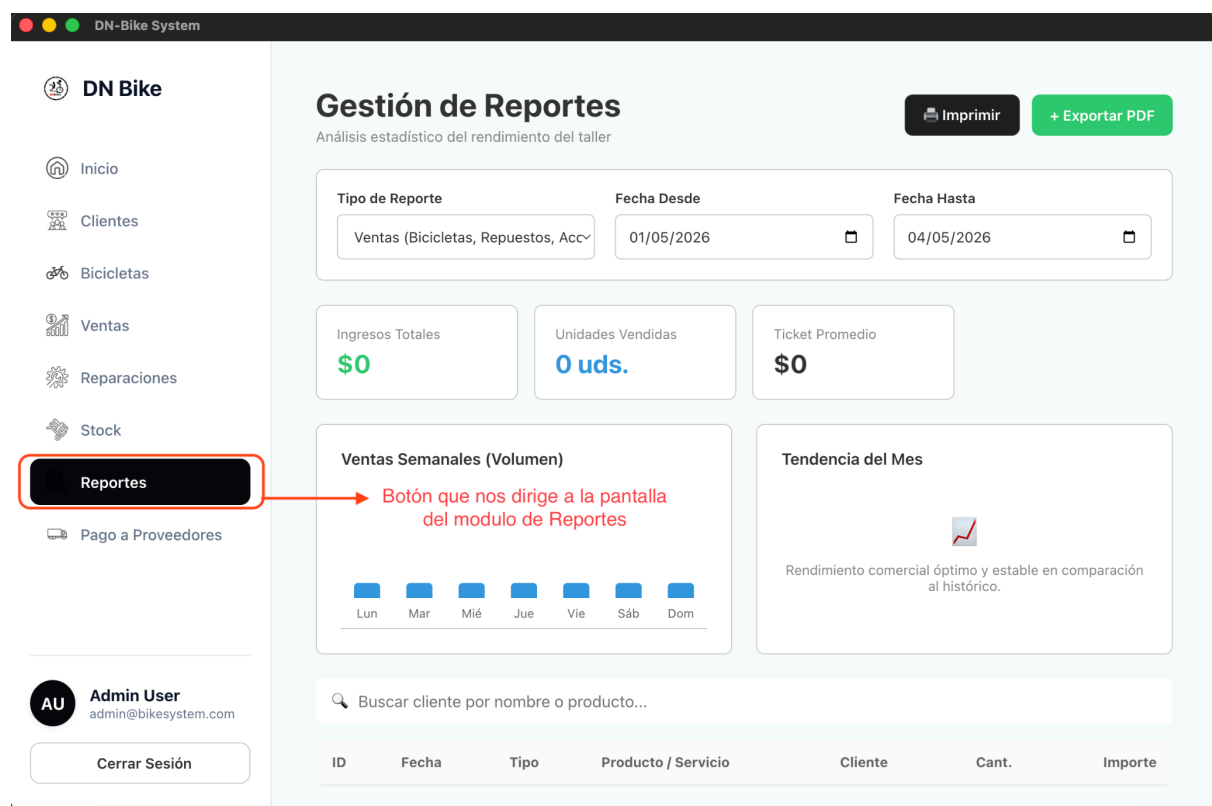


Figura 12: Pantalla del módulo de Gestión de Reportes.

Módulo pago proveedores.

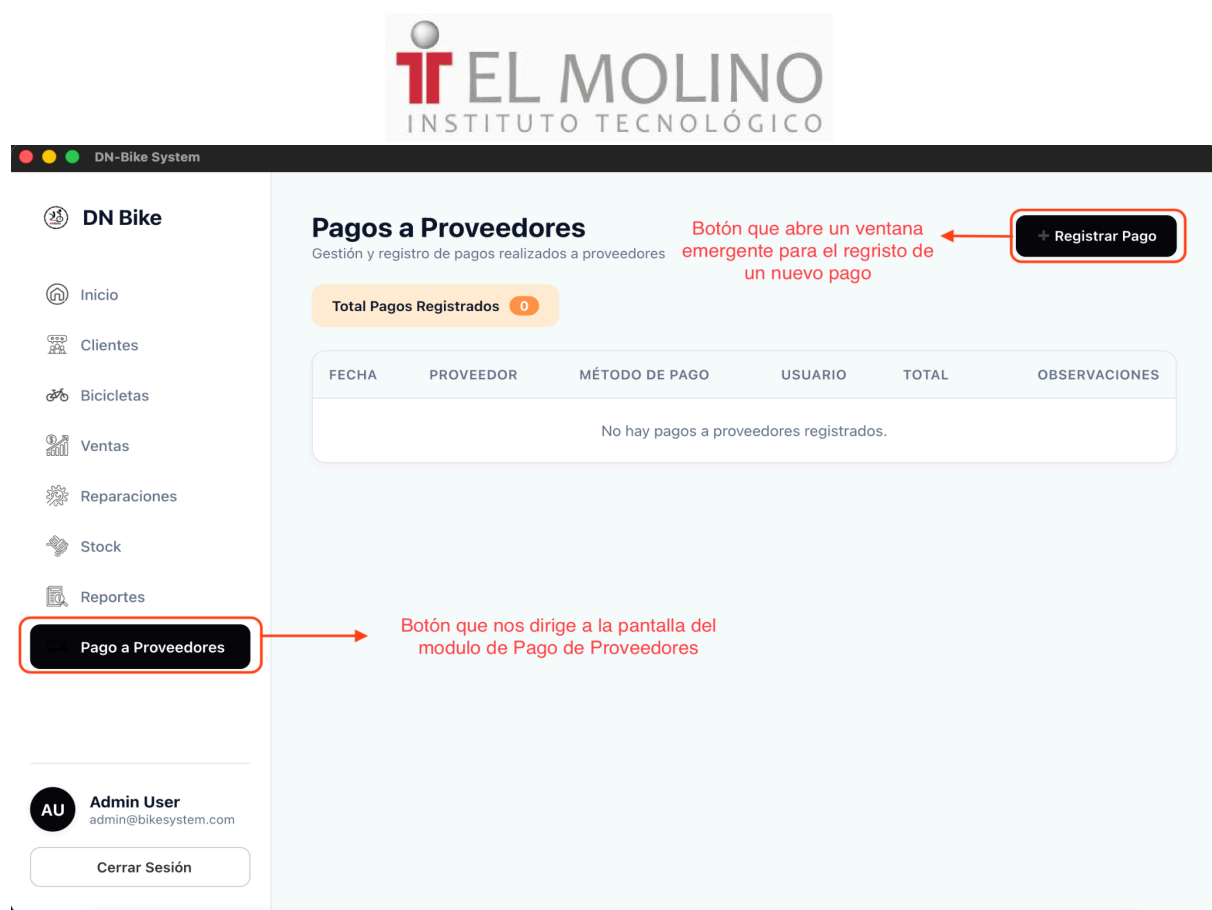


Figura 13: Pantalla del módulo de Pagos a Proveedores.

Ventana emergente pago proveedores

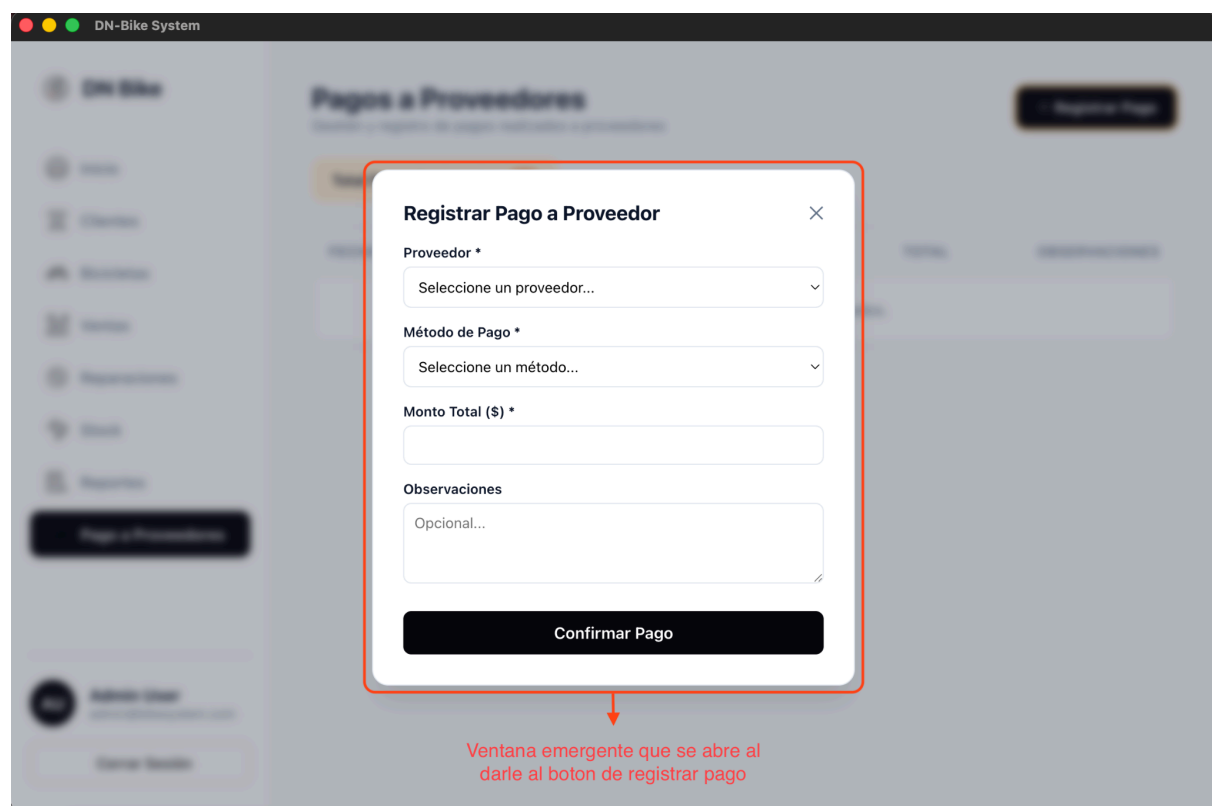


Figura 14: Ventana emergente para registrar un pago a proveedor.

Nota: aclaramos que esta ya son las pantallas definidas en el código y que no es un boceto o maqueta.


```
id_usuario INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
nombre_usuario VARCHAR(50) NOT NULL UNIQUE,  
contrasena VARCHAR(255) NOT NULL,  
rol VARCHAR(20) NOT NULL  
);  
  
CREATE TABLE Cliente (  
    id_cliente INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    apellido VARCHAR(100) NOT NULL,  
    dni VARCHAR(20) UNIQUE,  
    telefono VARCHAR(20),  
    email VARCHAR(100),  
    direccion VARCHAR(200)  
);  
  
CREATE TABLE Proveedor (  
    id_proveedor INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nombre_empresa VARCHAR(100) NOT NULL,  
    cuit VARCHAR(20) UNIQUE,  
    telefono VARCHAR(20),  
    email VARCHAR(100),  
    direccion VARCHAR(200)  
);  
  
CREATE TABLE Productos (  
    id_producto INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    marca VARCHAR(50),  
    modelo VARCHAR(50) NULL,  
    tipo_prod VARCHAR(50) NOT NULL,
```

```
cantidad INT DEFAULT 0,
numero_serie VARCHAR(100) NULL,
color VARCHAR(30) NULL,
rodado VARCHAR(20) NULL,
talle VARCHAR(20) NULL,
precio DECIMAL(10, 2),
stock_minimo INT DEFAULT 0
);

CREATE TABLE Metodo_Pago (
    id_metodo_pago INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

INSERT INTO Metodo_Pago (nombre) VALUES
('Efectivo'),
('Transferencia Bancaria'),
('Cheque'),
('Mercado Pago');

CREATE TABLE Bicicleta (
    id_bicicleta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_cliente INT NOT NULL,
    marca VARCHAR(50),
    modelo VARCHAR(50),
    CONSTRAINT fk_bici_cliente FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)
    ON DELETE RESTRICT
);

CREATE TABLE Venta (
    id_venta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_usuario INT NOT NULL,
```

fecha DATE NOT NULL DEFAULT CURRENT_DATE,

costo_total DECIMAL(10, 2) NOT NULL DEFAULT 0,

CONSTRAINT fk_venta_cliente FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)
ON DELETE RESTRICT,

CONSTRAINT fk_venta_usuario FOREIGN KEY (id_usuario) REFERENCES
Usuario(id_usuario) ON DELETE RESTRICT

);

CREATE TABLE Reparacion (

id_reparacion INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_bicicleta INT NOT NULL,

id_usuario INT NOT NULL,

fecha_ingreso DATE NOT NULL DEFAULT CURRENT_DATE,

fecha_egreso DATE,

estado VARCHAR(50) NOT NULL,

descripcion TEXT,

costo_mano_obra DECIMAL(10, 2) DEFAULT 0,

costo_total DECIMAL(10, 2) DEFAULT 0,

CONSTRAINT fk_rep_bicicleta FOREIGN KEY (id_bicicleta) REFERENCES
Bicicleta(id_bicicleta) ON DELETE RESTRICT,

CONSTRAINT fk_rep_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario)
ON DELETE RESTRICT

);

CREATE TABLE Ingreso_Stock (

id_ingreso INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_proveedor INT NOT NULL,

id_usuario INT NOT NULL,

fecha_ingreso DATE NOT NULL DEFAULT CURRENT_DATE,

num_comprobante VARCHAR(100),

CONSTRAINT fk_ingreso_prov FOREIGN KEY (id_proveedor) REFERENCES
Proveedor(id_proveedor) ON DELETE RESTRICT,

```
CONSTRAINT fk_ingreso_usuario FOREIGN KEY (id_usuario) REFERENCES  
Usuario(id_usuario) ON DELETE RESTRICT
```

```
);
```

```
CREATE TABLE Pago_Proveedor (
```

```
    id_pago INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    id_proveedor INT NOT NULL,
```

```
    id_usuario INT NOT NULL,
```

```
    id_metodo_pago INT NOT NULL,
```

```
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
```

```
    monto_total DECIMAL(10, 2) NOT NULL,
```

```
    observaciones TEXT,
```

```
    CONSTRAINT fk_pago_prov FOREIGN KEY (id_proveedor) REFERENCES  
Proveedor(id_proveedor) ON DELETE RESTRICT,
```

```
    CONSTRAINT fk_pago_usuario FOREIGN KEY (id_usuario) REFERENCES  
Usuario(id_usuario) ON DELETE RESTRICT,
```

```
    CONSTRAINT fk_pago_metodo FOREIGN KEY (id_metodo_pago) REFERENCES  
Metodo_Pago(id_metodo_pago) ON DELETE RESTRICT
```

```
);
```

```
CREATE TABLE Detalle_Venta (
```

```
    id_detalle_venta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    id_venta INT NOT NULL,
```

```
    id_producto INT NOT NULL,
```

```
    cantidad INT NOT NULL CHECK (cantidad > 0),
```

```
    precio_unitario DECIMAL(10, 2) NOT NULL,
```

```
    costo_total DECIMAL(10, 2) NOT NULL,
```

```
    CONSTRAINT fk_detventa_venta FOREIGN KEY (id_venta) REFERENCES Venta(id_venta)  
ON DELETE CASCADE,
```

```
    CONSTRAINT fk_detventa_prod FOREIGN KEY (id_producto) REFERENCES  
Productos(id_producto) ON DELETE RESTRICT
```

```
);
```

```
CREATE TABLE Detalle_Reparacion (
```

```

id_detalle_rep INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_reparacion INT NOT NULL,

id_producto INT NOT NULL,

cantidad INT NOT NULL CHECK (cantidad > 0),

precio_unitario DECIMAL(10, 2) NOT NULL,

costo_total DECIMAL(10, 2) NOT NULL,

        CONSTRAINT fk_detrep_rep FOREIGN KEY (id_reparacion) REFERENCES
Reparacion(id_reparacion) ON DELETE CASCADE,

        CONSTRAINT fk_detrep_prod FOREIGN KEY (id_producto) REFERENCES
Productos(id_producto) ON DELETE RESTRICT

);

CREATE TABLE Detalle_Ingreso (

    id_detalle_ing INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    id_ingreso INT NOT NULL,

    id_producto INT NOT NULL,

    cantidad INT NOT NULL CHECK (cantidad > 0),

    precio_costo DECIMAL(10, 2) NOT NULL,

        CONSTRAINT fk_deting_ingreso FOREIGN KEY (id_ingreso) REFERENCES
Ingreso_Stock(id_ingreso) ON DELETE CASCADE,

        CONSTRAINT fk_deting_prod FOREIGN KEY (id_producto) REFERENCES
Productos(id_producto) ON DELETE RESTRICT

);

-- 1. Creamos la función (Procedimiento Almacenado)

CREATE OR REPLACE FUNCTION actualizar_stock_venta()

RETURNS TRIGGER AS $$

BEGIN

    -- Descontar la cantidad vendida del stock del producto

    UPDATE Productos

    SET cantidad = cantidad - NEW.cantidad

    WHERE id_producto = NEW.id_producto;

```

```
RETURN NEW;  
  
END;  
  
$$ LANGUAGE plpgsql;  
  
-- 2. Creamos el Trigger  
  
CREATE TRIGGER trg_actualizar_stock_venta  
AFTER INSERT ON Detalle_Venta  
FOR EACH ROW  
EXECUTE FUNCTION actualizar_stock_venta();
```